Claude Code: 架构与实现

深入解析 Anthropic 的 AI 编程助手

自动生成的技术文档

2026 年 4 月 1 日

目录

第一章　应用程序引导与入口点

一个健壮的命令行应用程序的启动序列很少是一个简单的从上到下的脚本。对于 Claude Code 来说，引导（bootstrapping）是一个经过精心编排的过程，它在瞬时反馈的需求与初始化丰富运行时环境的复杂性之间取得了平衡。本章将探讨 Claude Code 如何管理其入口点、路由用户命令、协商网络传输，并建立所有后续操作所需的全局状态。

1.1　入口点与 SDK 接口

`entrypoints/` 目录充当 Claude Code 的公共 API 层面。传统的 CLI 通常假设总是有人类用户坐在终端前，而 Claude Code 的设计旨在在多种上下文中运行：作为交互式终端应用程序、作为无头模型上下文协议（MCP）服务器，以及作为外部 SDK（如 Python 和 Rust）的基础引擎。为了在不增加任何单一用例启动时间的情况下支持这些不同的目标，该应用程序在其最边缘处采用了一种专门的路由模式。

主要的入口点是 `cli.tsx`。该文件充当高速路由器。在加载任何复杂的应用逻辑、React 组件或 API 客户端之前，`cli.tsx` 会解析 `process.argv` 以识别请求的操作。它为特定的标志和子命令实现了“快速路径”(fast-paths)。例如，执行 `claude --version` 除了版本字符串本身之外，不需要导入任何模块，从而使其能够在几毫秒内打印并退出。这部分是通过利用构建时优化（例如 `bun:bundle` 指令）来执行基于特性开关的代码消除来实现的。

当检测到标准交互式命令时，`cli.tsx` 会推迟进入次要初始化阶段；但如果请求了替代的运行时目标，路由器则会完全走另一条分支。其中一个分支就是 MCP 服务器模式（`mcp.ts`）。当以此模式调用时，Claude Code 将作为一个独立的进程运行，严格通过标准输入和输出（STDIO）进行通信。该模式剥离了所有交互式 UI 组件，建立了一个专门为外部 MCP 客户端设计的隔离执行上下文。它维护着自己的内部 LRU 文件缓存（上限为 100 个条目），并将内部的 Zod 验证模式（schemas）自动转换为 MCP 规范所需的标准 JSON Schema 输出。

1.1.1　Schema 优先的 SDK 基础

入口点架构的很大一部分是专门为 Agent SDK 设计的。Claude Code 并没有为每种支持的语言手工编写类型定义，而是在 `entrypoints/sdk/` 中采用了 Schema 优先的设计。在这里，Zod 模式作为应用程序数据结构的无可争议的唯一可信数据源（source of truth）。

coreSchemas.ts 文件定义了所有可序列化的类型，包括配置、MCP 服务器设置以及 30 多种不同的生命周期钩子事件。这些 Zod 模式被输入到一个自动生成管道（scripts/generate-sdk-types.ts）中，由该管道生成并由 coreTypes.ts 导出 TypeScript 类型。

```
// Example of the SDK Control Protocol Schema definition pattern
export const SDKControlRequestSchema = z.discriminatedUnion("subtype", [
  InitializeRequestSchema,
  PermissionRequestSchema,
  McpMessageSchema
  // ...
]);
```

为了促进 Node.js CLI 进程与外部 SDK 实现之间的通信，controlSchemas.ts 定义了双向的进程间通信（IPC）协议。SDK 消费者通过标准输入发送 JSON 序列化的控制请求，CLI 在处理和响应之前会根据 SDKControlRequestSchema 对其进行验证。为了防止核心数据类型与控制协议类型之间出现循环依赖问题，该架构严重依赖于 lazySchema() 包装器，确保模式仅在验证期间明确需要时才会被求值。

此外，面向公众的 agentSdkTypes.ts 文件提供了一套存根函数（stub functions）。对于在 IDE 或外部脚本中集成 Claude Code 的开发者来说，这些强类型的存根记录了预期的 API 层面。实际的执行逻辑深深隐藏在宿主 CLI 中，但这一边界确保了类型安全、出色的 IDE 自动补全功能，并为 SDK 消费者提供了面向未来的保障。

1.2 CLI 路由与 I/O 传输

一旦 cli.tsx 确定用户正在调用标准的 CLI 命令，执行权就会移交给 cli/ 目录。这一层负责命令路由、I/O 编排和传输协商。它掌控着初始启动之后、核心查询执行引擎接管之前发生的一切。

1.2.1 延迟加载的命令处理器

Claude Code 中的命令被结构化为独立的处理器，位于 cli/handlers/ 目录中（例如 auth.ts、agents.ts、mcp.ts、plugins.ts）。这里一个关键的架构决策是使用动态导入。通过延迟加载这些处理器，在输入 claude help 或调用简单的实用命令时，就能避免因解析整个应用程序而产生的内存和 CPU 开销。

尽管它们是相互隔离的，但所有处理器都共享一致的执行模式。它们利用共享的服务层来进行分析、配置解析和错误格式化。此外，所有处理器都通过一个集中的退出实用程序（cli/exit.ts）终止。通过强制所有命令使用 cliOk() 或 cliError()，而不是将 process.exit() 调用散布在代码库各处，系统确保了一致的输出格式、可靠的遥测数据刷新以及更好的可测试性。

1.2.2 结构化与远程 I/O

CLI 操作的核心是 `print.ts`，它包含中央的读取-求值-输出循环（REPL）。这个循环完全与传输协议无关。它管理会话状态、工具执行和 MCP 服务器生命周期，而无需知道用户是在本地键盘上打字还是从云环境连接的。

为了将这个与传输无关的循环与外部世界连接起来，Claude Code 实现了两条不同的 I/O 路径：

- **结构化 I/O（`structuredIO.ts`）**：用于本地执行，通过 STDIO 进行通信。它读取并写入 NDJSON（换行符分隔的 JSON）控制消息。
- **远程 I/O（`remoteIO.ts`）**：在以 SDK 模式运行或连接到 Cloud Code Runtime (CCR) 时使用。它基于 HTTP/SSE 或 WebSockets 运行。

I/O 层中一个值得注意的工程细节是依赖于安全的 NDJSON 序列化器（`ndjsonSafeStringify.ts`）。虽然原始的 U+2028（行分隔符）和 U+2029（段落分隔符）字符在 JSON 字符串中是完全合法的，但当被 JavaScript 的字符串分割逻辑解析时，它们会充当实际的换行符。为了防止这些字符破坏 NDJSON 流并导致外部消费者崩溃，所有数据在跨越 I/O 边界之前，都要经过严格的转义函数处理。

1.2.3 权限竞速与传输多态性

在混合的本地/远程 AI 工具中，最复杂的用户体验挑战之一就是处理权限。Claude Code 在 `structuredIO.ts` 内部采用了“权限竞速”（Permission Racing）架构。当某个工具需要授权（例如，读取受保护的文件）时，系统会同时触发本地权限钩子（如根据配置文件自动确认特定目录的读取），并发送一个远程 `control_request` 请求人类批准。无论哪个先解决就赢得竞速，而失败者会立即被中止。这赋予了本地规则超低的延迟，同时也能优雅地降级为人工干预。

对于远程执行，Claude Code 在 `cli/transports/` 中实现了一个高弹性、运行时多态传输层。工厂函数 `getTransportForUrl()` 会检查环境变量和目标 URL 方案（scheme），以实例化以下三种传输之一：

1. **WebSocket 传输**：实时双向通信的默认方式。它具有用于重新连接的自动指数退避（exponential backoff）、10 分钟的重连预算（允许服务器优雅地终止空闲会话），以及防止主动代理超时的 ping/pong 保活机制。
2. **SSE 传输**：使用服务器发送事件（Server-Sent Events）进行长轮询读取，并使用批量 HTTP POST 进行写入。
3. **混合传输**：结合 WebSockets 用于低延迟读取，以及批量 HTTP POST 用于写入。

   架构说明：防止 Firestore 冲突
   SSE 和混合传输都严重依赖于 `SerialBatchEventUploader`。由于远程会话数据经常同步到云数据库（如 Firestore），向同一个文档发出的并发 POST 请求会相互覆

盖。上传器将所有传出事件强制放入一个严格的序列化队列中，将它们批量打包（每个 POST 最多 500 个项目）并执行背压（backpressure），从而保证数据的完整性。

1.3 全局状态初始化

虽然 CLI 路由器决定了执行权要转移到 *哪里*，但应用程序仍必须设置其运行的 *环境*。这分为两个领域：在 `init.ts` 中设置环境，以及在 `bootstrap/state.ts` 中进行内存状态管理。

1.3.1 预检初始化阶段

在需要完整的 CLI 模块（`cli/main.js`）之前，`cli.tsx` 会调用 `init()` 函数。这个预检（pre-flight）阶段对于建立应用程序的安全边界至关重要。它会验证用户配置文件、配置内部网络代理，并设置企业环境所需的 mTLS（双向 TLS）证书。

关键的是，`init.ts` 处理了重型依赖项的延迟加载。例如，初始化 OpenTelemetry 指标和追踪管道会占用大量的内存（约 400KB）。通过将此初始化推迟到主循环开始之前，应用程序保留了快速实用命令所需的超快启动时间。此外，`init.ts` 在启动序列中散布了大量 `profileCheckpoint()` 调用，使维护人员在调试启动缓慢问题时能够生成细粒度的启动性能火焰图。

1.3.2 中央 STATE 单例

传统的命令行应用程序通常在状态管理上苦苦挣扎，这往往导致深层嵌套的 Props 透传（prop-drilling）或混乱的全局变量。Claude Code 通过在 `bootstrap/state.ts` 中建立一个严格且类型安全的单例（singleton）解决了这个问题。

`State` 接口是运行时数据流的支柱。它包含超过 80 个不同的字段，跟踪从 API 成本指标和模型使用情况到活跃会话 UUID、插件注册表以及类似 cron 的后台任务等一切内容。当引导阶段调用 `getInitialState()` 时，它会读取当前工作目录（CWD），解析任何符号链接以建立规范的执行路径，并为会话生成一个唯一的 UUID。

为了防止意大利面条式的代码修改，`state.ts` 强制使用严格的访问器模式。模块不能随意覆盖状态字段；它们必须使用明确的、类型安全的 setter 函数。

```
// Example of the safe state mutation pattern
export function addModelUsage(model: string, tokens: number): void {
  const currentState = STATE.modelUsage.get(model) || 0;
  STATE.modelUsage.set(model, currentState + tokens);
  // Emit telemetry or trigger reactive updates as needed
}
```

`STATE` 单例也与应用程序的遥测优先（telemetry-first）设计紧密相连。在初始化期间，OpenTelemetry 的 `Meter`、`Logger` 和 `Tracer` 提供程序被直接附加到全局状态对象上。这确保了任何模块，无论它在工具执行或插件钩子中的嵌套层级有多深，都可以即时访问活跃的 Span 并发出结构化日志。

此外，状态系统还处理“粘性”会话行为。Beta 特性标志（例如 `afkMode` 或 `fastMode`）作为标头锁存器（header latches）实现。一旦通过命令行标志或提示词指令激活，状态单例会在该会话 UUID 的持续时间内将其锁定。这防止了不稳定行为的发生，比如提示词缓存可能在会话中途突然失效，仅仅是因为后续命令忘记传递正确的 CLI 标志。通过这一稳健的引导序列，Claude Code 确保在用户输入他们的第一条提示词时，一个具有弹性、类型安全且高度插桩检测的运行时环境已经准备好接收它。

第二章　核心数据结构与状态管理

2.1　中央类型系统与验证模式 (Schemas)

Claude Code 建立在严格、全面的 TypeScript 定义和运行时验证模式（Schema）的基础之上。由于该应用程序需要编排外部插件、复杂的本地 Hook 以及异步的语言模型交互，因此仅依赖编译时检查是远远不够的。该架构将其契约层分为编译时类型（存放在 `types/` 目录中）和运行时验证模式（在 `schemas/` 目录中管理）。

2.1.1　基于 Brand 模式的名义类型标识符

在分布式系统和复杂状态机中，隐蔽错误的最常见来源之一就是标识符混淆——例如在需要会话（Session）ID 的地方意外传入了用户（User）ID，因为两者通常都是字符串。Claude Code 完全通过使用 TypeScript 的"品牌类型（Branded Types）"模式实现名义类型（Nominal Typing）来彻底消除这一问题。

在 `types/ids.ts` 中定义，诸如 `SessionId` 和 `AgentId` 等标识符在编译时属于截然不同的类型：

```
export type SessionId = string & { readonly __brand: unique symbol };
export type AgentId = string & { readonly __brand: unique symbol };

export function toAgentId(id: string): AgentId {
  if (!/^a(?:.+-)?[0-9a-f]{16}$/.test(id)) {
    throw new Error(`Invalid Agent ID format: ${id}`);
  }
  return id as AgentId;
}
```

这种严格的类型控制贯穿于整个执行路径。当文本输入触发命令时，`AgentId` 会被附加到队列负载（Payload）中。当 Hook 回调触发或生成会话日志时，`SessionId` 会将这些分析数据关联起来。编译器强制保证 `AgentId` 绝不会意外地交叉污染需要 `SessionId` 的负载数据。

2.1.2 用于详尽匹配的辨识联合 (Discriminated Unions)

为了消除“字符串风格类型（stringly-typed）”的错误处理机制和脆弱的 switch 语句，系统高度依赖辨识联合（Discriminated Unions）。例如，插件系统定义了一个 `PluginError` 联合类型，包含 24 种不同的错误变体，每种变体都有特定的元数据。同样，Hook 事件系统使用包含 17 种 Hook 事件类型的辨识联合。

通过对共享的辨识字段（例如 `type`）进行 `switch` 判断，开发者可以充分利用 TypeScript 的详尽控制流分析。如果联合类型中新增了一个 Hook 类型或错误变体，编译器会立即标记出整个 monorepo 中遗漏的 `case` 语句，从而确保在不依赖运行时回退（Fallback）的情况下，对所有边界情况实现稳健的覆盖。

2.1.3 运行时验证与惰性 Schema 模式

虽然 TypeScript 处理了内部边界，但外部输入——如插件清单、模型上下文协议（MCP）服务器响应和配置文件——必须在运行时进行验证。Claude Code 使用 Zod 来保证这种结构完整性。

在大型 Zod 代码库中，一个主要的架构挑战是模块间的循环依赖：Schema 通常需要引用 TypeScript 接口，而这些接口反过来又使用 `z.infer<…>` 从其他 Schema 派生其类型。为了解决这个问题，Claude Code 采用了 *惰性 Schema*（*Lazy Schema*）模式。在 `schemas/hooks.ts` 中可以看到，Schema 被包裹在工厂函数或 Zod 的 `z.lazy()` 抽象中，以将计算推迟到该 Schema 被实际调用时进行。

```
export const buildHookSchemas = () => {
  return z.lazy(() => z.discriminatedUnion("type", [
    BashCommandHookSchema,
    PromptHookSchema,
    HttpHookSchema,
    AgentHookSchema
  ]));
};
```

2.1.4 规范的时间表示

对于数据分析、日志记录和远程同步而言，时间精度至关重要。由于浮点数的限制，JavaScript 原生的 `number` 类型在表示纪元纳秒（Epoch Nanoseconds）时会丢失精度。Claude Code 通过将所有时间数据标准化为 Google 的通用类型（Well-Known Type, WKT）`google.protobuf.Timestamp` 来解决这个问题。

在 `types/generated/google/timestamp.ts` 中，这段自动生成的 protobuf 代码使用了一个 64 位的 `Long` 抽象来表示秒（防止在 2262 年之后发生溢出），并使用一个单独的 32 位整数来表示

纳秒。代码库中的每个事件 Schema 都会委托该单一文件中的 `toJSON()` 和 `fromJSON()` 方法进行处理，从而建立一种不可破坏的、规范的时间传输格式（Wire Format）。

2.2 不可变配置与限制

一个稳健的 CLI 工具必须严格管理系统边界、API 限制和功能开关。在 Claude Code 中，`constants/` 目录作为这些边界的唯一、不可变的真相源（Source of Truth）。在设计上，该目录完全不包含任何依赖项。`constants/` 中的模块不会从应用程序的其余部分导入任何内容，从而消除了循环依赖的风险，并允许任何层（从最底层的解析器到最高层的 UI React 组件）安全地使用它们。

2.2.1 死代码消除与打包优化

为了保持 CLI 的响应速度并最小化占用空间，配置层针对构建时的摇树优化（Tree-shaking）进行了优化。文件大量使用 TypeScript 的 `as const` 断言，并导出扁平的字典或命名的原始值（Primitives）。

因为这些常量是可以静态分析的，打包工具可以在生产环境构建中积极地剔除未使用的数据。如果 `turnCompletionVerbs.ts` 中的某个特定的 UI 动词，或者 `xml.ts` 中的某个特定的 XML 标签在最终的依赖图中没有被引用，它会在编译期间直接被消除。

2.2.2 双层限制：客户端与 API

Claude Code 明确区分了 *客户端主动限制与 API 强制边界*。

`apiLimits.ts` 文件定义了 Anthropic API 的硬性约束（例如，每张图像最大 5MB，每个 PDF 最多 100 页，或每次请求最多 100 个媒体项）。这些值被预检验证逻辑用来实现快速失败，并在发起网络请求之前提供本地的、具有可操作性的错误信息。

相反，`toolLimits.ts` 定义了灵活的、客户端的应用程序约束。例如，工具结果字符串默认截断为 50,000 个字符，每个消息轮次的合计字符预算为 200,000 个字符。这使得系统的 `systemPrompts.ts` 缓存逻辑能够抢先截断冗长的本地工具输出（比如庞大的 `grep` 结果），以维持上下文窗口的稳定性，而不会触碰 API 的硬性限制。

2.2.3 功能门控与 Beta 注册表

实验性功能通过 `betas.ts` 和 `tools.ts` 进行集中管理。功能的可用性通过一个集中的注册表进行门控（Gated），而不是将布尔值检查散布在整个代码库中。

这些文件利用 `feature()` 工具函数（通过环境绑定提供）来确定是否应暴露实验性功能，如快速模式（Fast Mode）、高级上下文解析或多智能体协调器。这种模式控制着向 API 请求中注入

自定义 HTTP 请求头，并过滤传递给智能体的允许工具集，从而允许针对特定的用户群体进行安全的、渐进式的发布部署。

2.2.4 治理与安全隔离

至关重要的是，安全策略与操作逻辑是解耦的。`cyberRiskInstruction.ts` 文件包含了核心的防护指令——这些规则定义了可接受的安全协助（如代码审计）与被禁止活动之间的边界。通过将这些指令隔离到一个专用的常量文件中，该架构确保了治理规则的任何变更都需要安全团队进行明确的、局部的审查，且完全独立于智能体本身的执行机制。

2.3 响应式状态与副作用

尽管 Claude Code 是一个命令行界面，但它的终端渲染是由 Ink（一个基于 React 的框架）驱动的。因此，它需要一个稳健的响应式状态管理架构。`state/` 目录实现了一种分层方法，将持久化的外部环境（磁盘上的配置文件）与短暂的、高度交互的内部 UI 状态桥接起来。

2.3.1 框架无关的 Store 基础

位于状态层级结构最底层的是 `store.ts`。Claude Code 没有引入像 Redux 或 Zustand 这样沉重的依赖，而是实现了一个极简的、与框架无关的 Store。它暴露了一个简单的契约接口：`getState()`、`setState()` 和 `subscribe()`。

对该 Store 的更新是严格不可变的。应用程序状态定义 `AppState`，是通过 `AppStateStore.ts` 中的 `getDefaultAppState()` 工厂函数初始化的。这种解耦使得核心状态引擎能够被后台进程、异步网络处理器或测试套件查询和操作，而无需依赖 React 的渲染上下文。

2.3.2 基于 Selector 的 React 集成

为了将原生 Store 桥接到 React UI 中，`AppState.tsx` 提供了一个 `AppStateProvider` 和一系列自定义 Hook。主要的 Hook `useAppState(selector)` 在底层利用了 React 的 `useSyncExternalStore`。

这种设计针对渲染性能进行了深度优化。通过向该 Hook 传递一个纯净的 Selector 函数（定义在 `selectors.ts` 中），组件只订阅它们关心的精确状态切片。状态对比使用了严格的 `Object.is` 相等性校验。如果全局状态更新了，但派生出来的切片保持不变，组件就会避免重新渲染。同样地，`useSetAppState` 返回的是状态修改器的稳定引用，这意味着只分发（Dispatch）动作的组件永远不会发生意外的重新渲染。

2.3.3 面向高频流式传输的推测状态 (Speculation State)

语言模型流式传输引入了一个独特的状态管理挑战：高频更新（通常每秒几十个 Token）会导致严重的 UI 卡顿，如果在收到每个 Token 时都对消息数组进行不可变的展开（Spread）和重新分配。

为了解决这个问题，`AppState` 引入了“推测状态”——在状态树内部为快速变化的数据嵌入可变的 `{ current: …}` React refs。这使得流引擎能够即时将 Token 追加到由 ref 支持的缓冲区中，同时 React 定期批量处理视觉更新，从而将网络分块（Chunk）速率与终端帧率解耦。

2.3.4 副作用的单一收口点

状态层中最关键的架构决策或许就是如何处理副作用。在一个标准的 React 应用程序中，副作用（如保存到磁盘或追踪分析数据）通常散布在各个组件的 `useEffect` Hook 中。

Claude Code 将所有状态转换汇聚到一个集中的单一收口点：`onChangeAppState.ts`。该模块被注册为底层 Store `onChange` 事件的唯一订阅者。每当 UI 状态发生突变时——无论是通过键盘快捷键、斜杠命令还是对话框确认——都由这个单一的处理器来处理状态转换。

它负责：

- 将 UI 开关状态同步回持久化的外部设置 Store 中。
- 将权限模式的更改（例如从“ask”切换到“auto”）通知给底层 SDK 和模型上下文协议（MCP）服务器。
- 当上下文边界发生改变时，使内存缓存失效。
- 在活动的子智能体会话间传播环境变量的更新。

通过集中处理副作用，该架构保证了不管用户交互是如何触发的，视觉状态和底层操作能力（如工具权限）永远不会出现同步脱节的情况。

结论

本书对 Claude Code 的架构与实现进行了全面的探讨。从权限系统和工具基础架构，到流式 I/O 处理与命令行（CLI）界面，我们从系统层面剖析了现代 AI 驱动的编码助手是如何构建的。

其模块化的设计、对 TypeScript 类型系统的广泛应用，以及通过权限管理对安全性的高度关注，充分展示了如何将软件工程的最佳实践应用于 AI 辅助开发这一充满挑战的领域。